

資料 v2.0

# ViennaRNA : RNA構造解析ソフトウェア

二次構造の予測から inverse folding（配列設計）までを体験する

分子ロボティクス夏の学校 2026

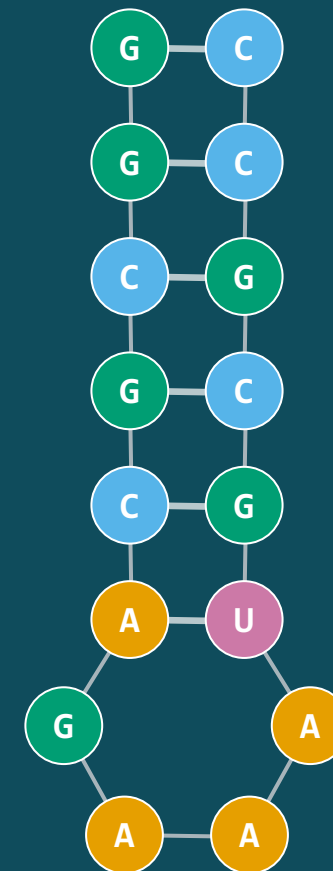
講師：角 俊輔

2026年10月8日（木）19:00–21:00（日本時間・Zoom）

使用ソフトウェア：ViennaRNA 2.7.2（Python から import RNA）

資料・演習スクリプト：[github.com/Shunsuke-1994/Course\\_RNAinverse](https://github.com/Shunsuke-1994/Course_RNAinverse)

ここからダウンロードしてそのまま使えます（Code → Download ZIP、または git clone）



GGCGCAGAAAUGCGCC

# 事前準備：実習環境のインストール（所要 10～15 分）



## 必要なもの

- 自分の PC（macOS または Windows）と管理者権限
- インターネット接続（ダウンロードは合計 100 MB 程度）
- ターミナル（macOS：ターミナル.app、Windows：PowerShell）
- exercises フォルダ（演習用スクリプト。GitHub から入手、手順 2・3 参照）
- Python の経験は不要。コマンドはコピーして貼り付ける

## 動作確認の状況

- 確認済み：macOS 15（Python 3.13.4）、Windows 11（Python 3.13.15）、いずれも viennarna 2.7.2
- Windows では演習2の設計配列が資料の例と変わる（乱数の違い。失敗ではない）
- 動かない場合：当日は TA またはグループの画面で結果を比較しながら参加できる。エラー文を控えておく

# 手順 1 : uv をインストールする

## macOS (ターミナル.app を開いて貼り付け)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

## 確認 (ターミナルを閉じて開き直してから)

```
uv --version  
# 出力例: uv 0.12.17 (...) ← バージョンが表示されれば OK
```

## Windows (PowerShell を開いて貼り付け)

```
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"
```

## uv とは

- Python 本体と仮想環境、パッケージをまとめて管理する軽量ツール
- 既存の Python 環境には手を加えない

## うまくいかないとき

- 「uv: command not found」→ ターミナルを開き直す。それでも出なければ macOS は ~/.local/bin/uv --version を試す
- Windows で実行ポリシーのエラー → 上のコマンドを PowerShell に 1 行のまま貼り付ける (-ExecutionPolicy ByPass が含まれていることを確認)
- すでに Python がある人は pip でも可 : python -m pip install uv (この場合も以降の手順は同じ)

# 手順 2・3 : プロジェクトを作り、ViennaRNA を入れる

```
# 手順 2: プロジェクトを作る (好きな場所で)
uv init viennarna-course --python 3.13
cd viennarna-course

# 手順 3: ViennaRNA (Python バインディング) を入れる
uv add viennarna==2.7.2

# exercises フォルダを GitHub から入手してここにコピー
# https://github.com/Shunsuke-1994/Course_RNAinverse
# 緑の Code ボタン → Download ZIP → 展開
# git を使う人は次の 1 行でも入手できる:
# git clone https://github.com/Shunsuke-1994/Course_RNAinverse.git
```

## 動作確認 (手順 4)

```
uv run python -c "import RNA;
print(RNA.__version__)"
# → 2.7.2

uv run python exercises/ex1_fold.py
# → ViennaRNA 2.7.2
# GCGCAGAAUGCGCC
# ((((((.....)))))) -10.20 kcal/mol
```

## フォルダ構成 (完成形)

```
viennarna-course/
  pyproject.toml  ← uv が作る (viennarna==2.7.2 が書かれる)
  .venv/         ← uv が作る仮想環境 (触らない)
  exercises/     ← GitHub の ZIP から取り出してコピー
```

## ポイント

- 以後は必ず `uv run python ...` で実行する (素の python では RNA が見つからない)
- 2.7.2 以外が表示されたら `uv add viennarna==2.7.2` をやり直す

# うまくいかないとき・uv を使わない代替手順

| 症状                                         | 原因の候補                     | 対応                                |
|--------------------------------------------|---------------------------|-----------------------------------|
| uv: command not found                      | PATH が未更新                 | ターミナルを開き直す／～/.local/bin/uv        |
| ModuleNotFoundError: No module named "RNA" | uv run を付けずに素の python を実行 | uv run python … で実行する             |
| No solution found / viennarna が見つからない      | Python のバージョン不一致、ネット未接続   | uv init --python 3.13 で作り直す。接続を確認 |
| バージョンが 2.7.2 でない                           | 別環境の ViennaRNA            | uv add viennarna==2.7.2 をやり直す     |
| exercises が見つからない                          | コピー先が違う                   | viennarna-course/exercises/ に置く   |
| Windows で文字化け                              | コンソールの文字コード               | 出力の数値と括弧だけ確認できれば OK               |

## uv を使わない代替（Python 3.10～3.14 が入っている場合）

```
python -m venv .venv
# macOS: source .venv/bin/activate      Windows: .venv/Scripts/activate
python -m pip install viennarna==2.7.2
python -c "import RNA; print(RNA.__version__)" # → 2.7.2
```

### 当日までに

- 動作確認の出力が出たら準備完了
- エラーが残る場合はエラー文を控えておく（当日 TA が対応）
- 動かなくても参加可。TA・グループの画面で結果を比較する

# 資料と演習スクリプトは GitHub に置いてあります

[https://github.com/Shunsuke-1994/Course\\_RNAinverse](https://github.com/Shunsuke-1994/Course_RNAinverse)

## 入手方法 A : ZIP でダウンロード（おすすめ）

緑の Code ボタン → Download ZIP → 展開し、中の exercises/ を viennarna-course/ にコピー（手順 2・3 のフォルダ構成）

## 入手方法 B : git clone（git を使える人）

```
git clone https://github.com/Shunsuke-1994/Course_RNAinverse.git
cd Course_RNAinverse
uv sync                                # Python 3.13 と viennarna 2.7.2 が入る
uv run python exercises/ex0_check.py
```

## リポジトリの中身

- 講義スライド（PDF、最新版）
- 事前準備の手順（PDF）
- exercises/ : 演習スクリプトと期待される出力
- README : 使い方の説明

## 質問は GitHub の Issues へ（講義後に対応します）

- リポジトリ上部の Issues タブ → New issue に質問を書く（GitHub アカウントが必要）
- エラーのときは、実行したコマンド・表示されたエラー文・OS を書くと答えやすい
- Issues は誰でも読める公開の場です。個人情報は書かないでください

# 今日の到達目標

## 知る

- RNA設計で「できること」の見取り図
- スイッチ・ナノ構造などの実例
- 予測と設計で何が分かり、何が分からないか

## 読む

- dot-bracket 表記で構造を読む
- 自由エネルギーの意味と大きさ
- MFE構造と構造集団（確率）の違い

## 動かす

- RNA.fold で構造を予測する
- 一塩基を変えて結果を比べる
- RNA.inverse\_fold で配列を設計し再予測・比較する

主目的ではないこと：動的計画法の証明、ソフトウェアの実装の理解、長いコードの手入力。必要な式と数値だけを扱い、細部は補足スライドとノートで支えます。

# 120分の進め方：説明と実習の時間配分

| 時刻          | 内容                                | 分  | 録画 |
|-------------|-----------------------------------|----|----|
| 19:00-19:12 | 到達目標とRNA設計の応用例                    | 12 | ●  |
| 19:12-19:30 | 二次構造・エネルギー・MFE・DP（動的計画法）の直感       | 18 | ●  |
| 19:30-19:45 | 演習1：構造予測と配列変更                     | 15 | 停止 |
| 19:45-20:00 | inverse folding と RNAinverse の仕組み | 15 | ●  |
| 20:00-20:40 | 演習2：配列設計・再予測・候補比較（グループ）           | 40 | 停止 |
| 20:40-20:50 | 応用への展開・まとめ・提出課題                   | 10 | ●  |
| 20:50-21:00 | 質疑応答                              | 10 | 停止 |

## 内訳

- 説明・まとめ 55分
- 実習 55分
- 質疑 10分

## 進め方

- 同じ16塩基RNAを使い続ける
- 短いコードを実行し、配列や目標構造だけを書き換える
- 演習はグループで、提出担当者の画面共有で進める
- 演習には「必須」「時間があれば」の表示あり

# RNA設計の応用：センサー、発現制御、触媒、ナノ構造

## センサー

- アプタマー：小分子やタンパク質に結合
- 結合で構造が変わり、蛍光や翻訳を切り替える

## 発現制御

- リボスイッチ（天然）、toehold switch（人工）
- 入力RNAの有無で翻訳ON/OFF

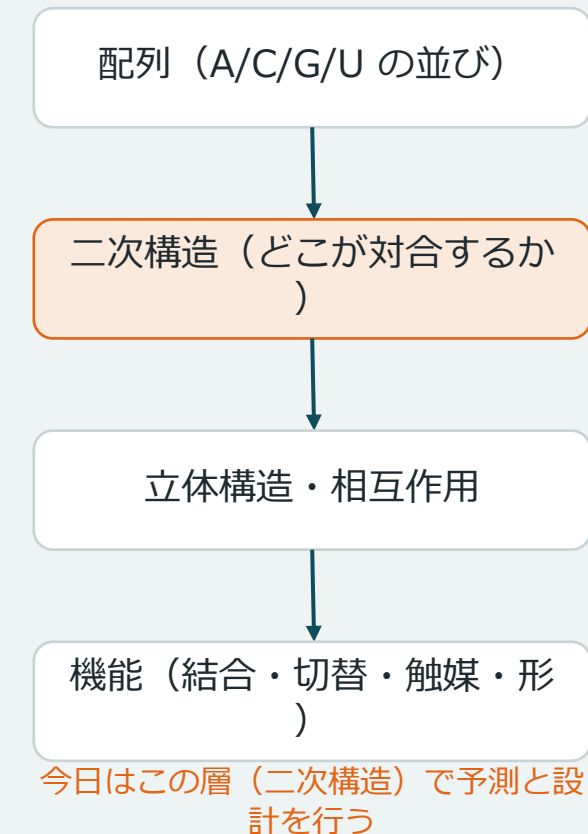
## 触媒

- リボザイム：RNA自身が反応を触媒
- ハンマーヘッド型、自己切断・連結

## ナノ構造

- RNA origami：1本鎖が転写中に折れてタイルに
- 分子の足場、細胞内で遺伝的に発現可能

## 共通の考え方



# 応用例：RNAスイッチ（toehold switch）は入力で働きを変える

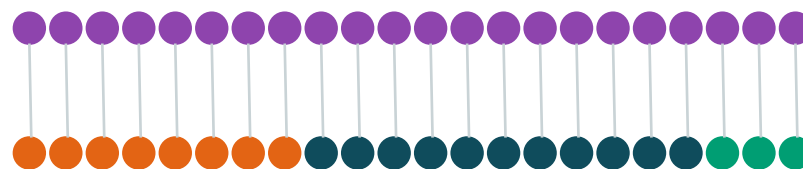
## OFF：スイッチRNA単独



- toehold（一本鎖）
- ステム
- RBS（ループ内に隠す）
- AUG（ステム内）
- 下流の遺伝子

## ON：トリガーRNAが結合して開く

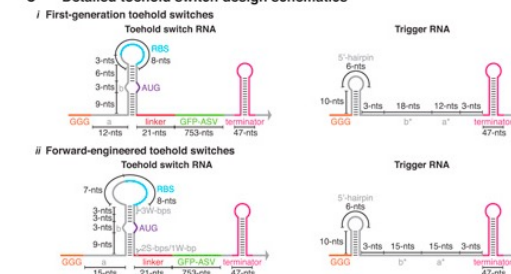
トリガーRNA



RBS と AUG が露出 → リボソームが結合 → 翻訳 ON

- スwitchRNA：ヘアピンで RBS と AUG を隠す（OFF）
- トリガーRNA：toehold に結合し鎖置換で開く（ON）
- 設計では OFF 構造と ON 複合体の両方を評価
- ON/OFF 比：第1世代平均 約43、改良版平均 406
- 直交セット：クロストーク12%未満が26系統

C Detailed toehold switch design schematics



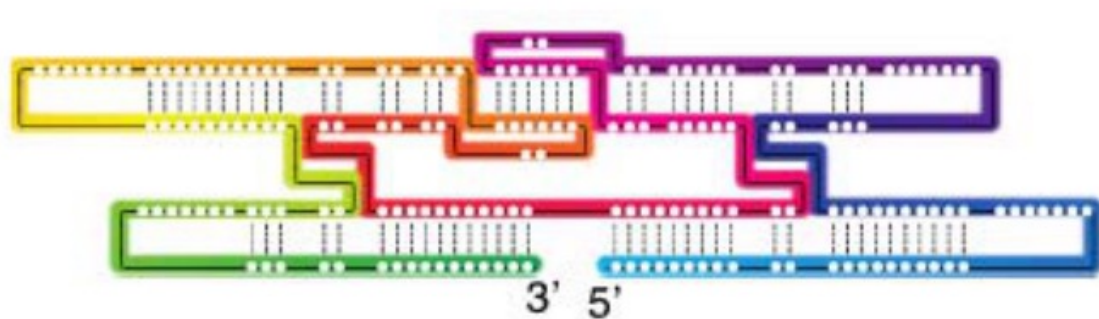
Green, Silver, Collins, Yin (2014) Cell 159:925–939. 図はPMC著者原稿版（PMC4265554）掲載図の一部（第1世代スイッチの設計模式図）

## ポイント

- 入力（トリガー）の有無で構造が変わる
- 「構造が変わる」を設計するには2状態の評価が要る（本日は単一状態のみ）

# 応用例：RNAナノ構造（RNA origami）と分子の配置

## 2H-AE



Geary, Rothmund, Andersen (2014) Science 345:799–804, Supplementary Fig. S4 より（2ヘリックススタイル 2H-AE のstrand-path図。色は5'→3'の合成順）

- 1本のRNA鎖が転写されながら折れる（cotranscriptional folding）
- らせんを架橋する crossover と、ループ同士が対合する kissing loop を組み合わせる
- タイルは最大 660 塩基。六角格子に自己集合
- 遺伝子として細胞内で発現できるのがDNAナノ構造との違い
- 二次構造予測は個々のステム・ヘアピン設計に使うが、kissing loop などの三次相互作用は別モデルが必要

1本鎖RNAを転写 → 転写中に折れる → タイル状の構造 → 格子に集合（AFMで観察）

本講習用の流れ図（模式）

# 今日扱う設計問題：単一RNAの目標二次構造に合う配列を探す

## 予測 (folding)

配列

GGCGCAGAAUGCGCC

↓ RNA.fold

(((((...)))))) -10.20 kcal/mol

## 設計 (inverse folding)

目標構造

(((((...))))))

↓ RNA.inverse\_fold

????????????

(複数の答え／答えなし)

- 入力：目標二次構造  $T$  (dot-bracket)。出力：予測構造が  $T$  に一致する配列  $x$
- 評価器は予測プログラムそのもの。「配列を少し変えて再予測」を繰り返す探索
- 距離  $d(\text{予測構造}, T)$  が 0 になれば成功。0 にならないこともある

## 今日は扱わない条件

- 複数の状態 (ON/OFF)
- 他の分子との相互作用
- 三次構造・擬似結び目
- 実験での機能確認

# RNAの配列と塩基対 : A-U、G-C、G-U



アデニン



シトシン



グアニン



ウラシル



水素結合 2本

標準 (Watson-Crick)



水素結合 3本

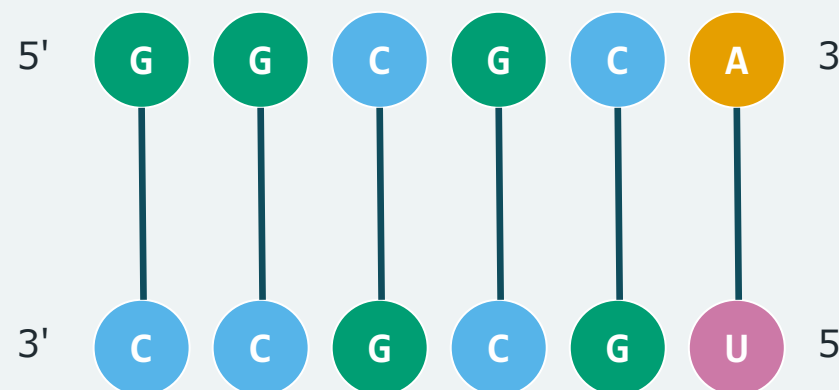
標準・最も安定



ゆらぎ対 (wobble)

弱いが許される

## 逆平行に並んで対合する



例題RNAの1-6番（上）と11-16番（下）。上の鎖は5'→3'、下の鎖は3'→5'に読む。1番のGは16番のCと、6番のAは11番のUと対合する。

塩基対の安定さ：水素結合 + 隣の塩基対との重なり (スタッキング)

# dot-bracket 表記 : 配列と構造を対応させて読む

1            5            10            15  
GGCGCAGAAUGCGCC

配列 (5'→3')

MFE = -10.20 kcal/mol

((((((((.....)))))))

構造 (dot-bracket)

読み方 : ( と ) は対合する2つの塩基、. は対合しない塩基。括弧は内側から順に対応 (入れ子)

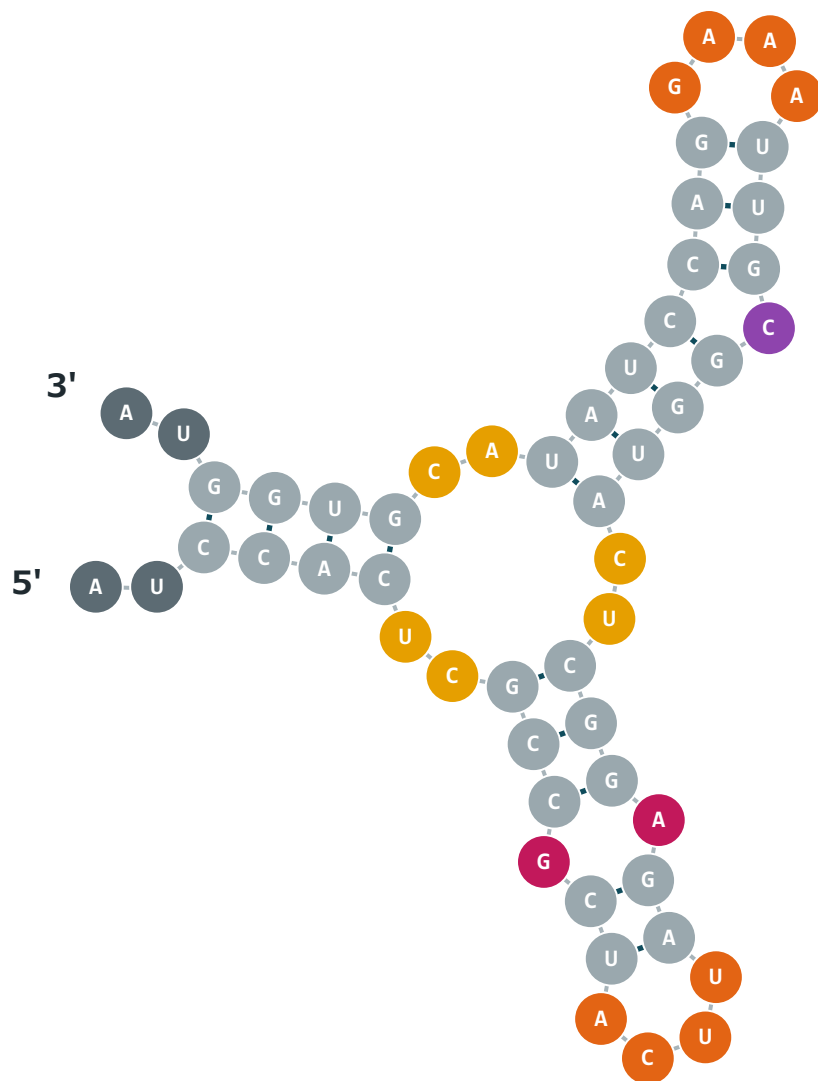
## 小問 (30秒)

- 3番目の C はどの番号の塩基と対合しているか
- ループ (対合しない部分) は何番から何番か
- 塩基対は何個あるか

## ヒント

- 括弧を内側から対応させる : 6番の ( と 11番の )
- その外側 : 5番と12番、4番と13番、...
- 対応する2文字は同じ色の位置で対合できる文字が確かめる (G-C, A-U, G-U)

# 二次構造の要素：ステム、ヘアピン、バルジ、内部ループ、分岐



● ステム (連続した塩基対)

●ヘアピンループ

● バルジ (片側のみ非対合)

● 内部ループ (両側に非対合)

● 多分岐ループ (3本のステムが集合)

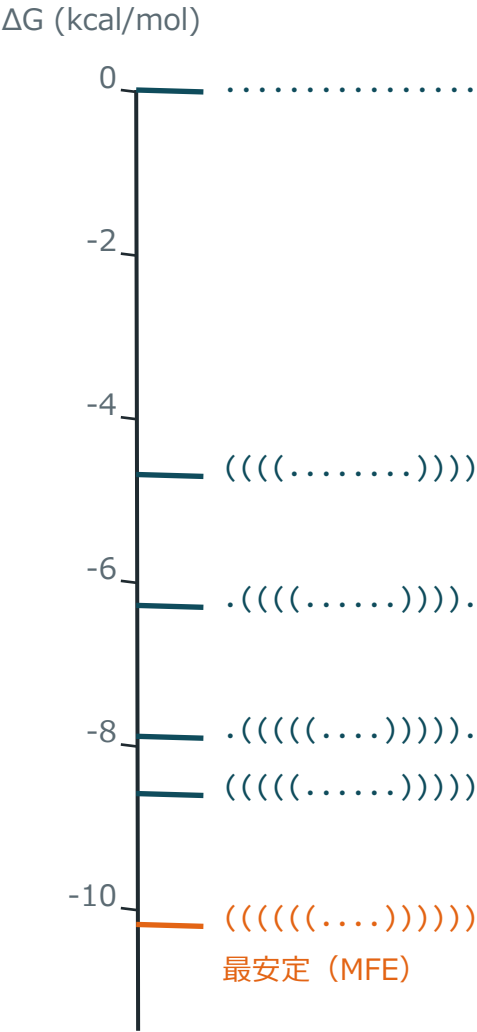
● 外部ループ (5'/3'末端側)

配列 (53塩基) と構造

```
AUCCACUCGCCGCUACUUAGAGGCUCA
UGGCGUUAAGGACCUAUACGUGGUA
..((((..(((.....)).)))..(
(((.....)))))).....
```

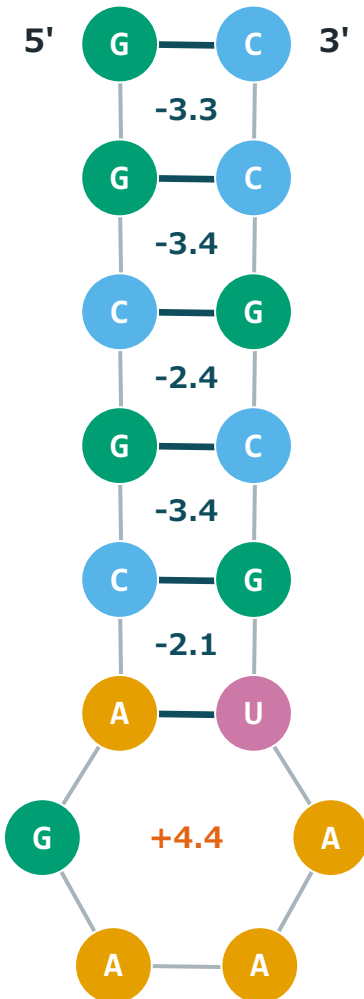
# 自由エネルギーと安定性：同じ条件で構造を比較する

| 構造（同じ配列 GGCGCAGAAAUGCGCC） | $\Delta G$<br>(kcal/mol) | 確率 P    |
|---------------------------|--------------------------|---------|
| (((((((.....)))))))))     | -10.2                    | 0.908   |
| (((((((.....)))))))))     | -8.6                     | 0.068   |
| .(((((((.....)))))))).    | -7.9                     | 0.022   |
| (((((((.....)))))))))     | -4.7                     | < 0.001 |
| .(((((((.....)))))))).    | -6.3                     | 0.002   |
| .....                     | 0.0                      | < 0.001 |



条件：37 °C、Turner 2004 パラメータ、ViennaRNA 2.7.2

# エネルギーモデル：スタッキングとループの寄与



$$E(S) = \sum_{L \in \text{loops}(S)} \Delta G(L)$$

構造  $S$  の自由エネルギーは、 $S$  をループに分解した各ループの寄与  $\Delta G(L)$  の和

| ループ    | 内容                  | $\Delta G$ |
|--------|---------------------|------------|
| スタッキング | (1,16) と (2,15)     | -3.3       |
| スタッキング | (2,15) と (3,14)     | -3.4       |
| スタッキング | (3,14) と (4,13)     | -2.4       |
| スタッキング | (4,13) と (5,12)     | -3.4       |
| スタッキング | (5,12) と (6,11)     | -2.1       |
| ヘアピン   | (6,11) が閉じる 4 塩基ループ | +4.4       |
| 合計     | MFE                 | -10.2      |

## 本数が同じでも違う

GGCGCAGAAAUGCGCC

(((((((.....)))))))))

6 対  $\Delta G = -10.2$  kcal/mol

UAUAUAGAAUAUAUA

(((((((.....)))))))))

6 対  $\Delta G = -1.2$  kcal/mol

GCGCGCGAAAGCGCGC

(((((((.....)))))))))

6 対  $\Delta G = -11.7$  kcal/mol

同じ形 (6対+4塩基ループ) でも、A-U だけなら -1.2、G-C だけなら -11.7

# MFE構造と構造集団：一つの予測構造だけでは分からないこと

## 例題RNA

弧の太さ = 塩基対確率  $p_{ij}$

## 2番目を G→C に変えた配列



GGCGCAGAAU GCGCC

(((((.....)))))) MFE 構造の確率  $P = 0.908$



GCCG CAGAAU GCGCC

..(((.....))).. MFE 構造の確率  $P = 0.378$

| 構造               | $\Delta G$ | P     |
|------------------|------------|-------|
| (((((.....)))))) | -10.2      | 0.908 |
| (((((.....)))))) | -8.6       | 0.068 |
| .(((.....))).    | -7.9       | 0.022 |
| .(((.....))).    | -6.3       | 0.002 |

| 構造                | $\Delta G$ | P     |
|-------------------|------------|-------|
| ..(((.....)))..   | -4.2       | 0.378 |
| ((.(((.....))))). | -3.9       | 0.233 |
| (.(((.....)))).   | -3.9       | 0.233 |
| (.(((.....)))).   | -3.0       | 0.054 |

$$P(S) = \frac{e^{-E(S)/RT}}{Z}, \quad Z = \sum_{S'} e^{-E(S')/RT}$$

構造全体の確率  $P(S)$  と、個々の塩基対の確率  $p_{ij}$  は別物：右では  $p(4,13)=0.997$  でも  $P(\text{MFE構造})=0.378$

# DP（動的計画法）の考え方：区間の計算結果を再利用する

## 候補の数（列挙は不可能）

| 配列長              | 構造の数                   |
|------------------|------------------------|
| 16（例題RNA、対合規則あり） | 347                    |
| 16（任意の対合）        | 5,223                  |
| 30（任意の対合）        | 約 $2.4 \times 10^8$    |
| 50（任意の対合）        | 約 $1.8 \times 10^{15}$ |
| 100（任意の対合）       | 約 $6 \times 10^{32}$   |

擬似結び目なし・最小ループ3塩基で数えた値

→ 短い区間の答えを使い回す

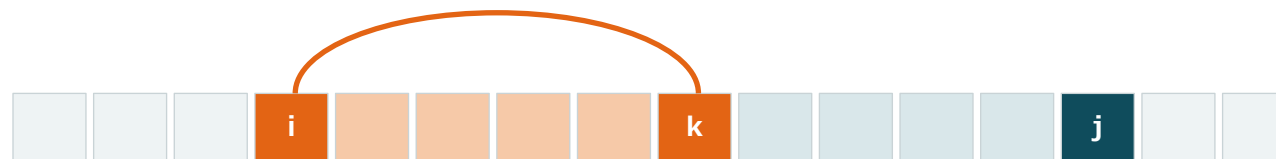
区間  $[i, j]$  の最良値は、より短い区間の最良値から決まる



区間  $[i, j]$ （求めたい）



場合1：i が対合しない →  $[i+1, j]$  の答えをそのまま使う



場合2：i が k と対合 → 内側  $[i+1, k-1]$  と外側  $[k+1, j]$  の答えを足す（k を全部試す）

短い区間から順に表を埋めると、長さ  $n$  の配列の最良構造が  $n^3$  程度の手間で求まる（ViennaRNA はループ単位のエネルギーで同じ考え方を使う）

注意：予測の DP を「逆向きに解く」と設計になるわけではない。設計は予測を評価器に使う探索（次の節）



## 録画を停止します：演習1（15分）

19:30–19:45 構造予測と配列変更（グループ演習）

提出担当者が画面共有しながら進めます。TA が各グループを支援します。

自分の環境が動かなくても、グループの画面を見て参加できます。

# 演習1の準備 : import とバージョン、入力配列の確認

ターミナルに入力する（この 1 行だけ）

```
uv run python exercises/ex0_check.py
```

コンピューターが返す結果（入力しない）

```
ViennaRNA 2.7.2  
配列: GGCGCAGAAAUGCGCC  
長さ: 16
```

exercises フォルダが手元にない人 : [github.com/Shunsuke-1994/Course\\_RNAinverse](https://github.com/Shunsuke-1994/Course_RNAinverse) から入手 (Code → Download ZIP、または次の git clone)

```
git clone https://github.com/Shunsuke-1994/Course_RNAinverse.git
```

## 確認すること

- エラーが出ずに 3 行表示される
- バージョンが 2.7.2（違えば TA に伝える）
- 長さが 16
- exercises フォルダの他のスクリプトも同じ形で実行する

## 動かないとき

- ModuleNotFoundError: No module named "RNA" → uv run を付け忘れたか、別のフォルダにいる。プロジェクトのフォルダで実行する
- >>> の画面になったら exit() で抜ける。どうしても動かない → グループの画面共有で参加する（結果の読み方は同じ）

# 演習1A : 短いRNAを fold し、構造とエネルギーを読む

```
import RNA
print("ViennaRNA", RNA.__version__)      # 2.7.2 と出れば OK

seq = "GGCGCAGAAAUGCGCC"                # 例題RNA (16 塩基)
ss, mfe = RNA.fold(seq)                  # 構造と最小自由エネルギー
print(seq)
print(ss, f"{mfe:.2f} kcal/mol")
```

期待する出力

```
ViennaRNA 2.7.2
GGCGCAGAAAUGCGCC
((((((...)))))) -10.20 kcal/mol
```

配布スクリプト : exercises/ex1\_fold.py

## 変更する入力

- 配列 seq (今回は例題 RNAのまま)

## 操作

- RNA.fold(seq) を実行
- 構造とエネルギーを表示

## 確認する出力

- dot-bracket 構造
- MFE (kcal/mol)

## 考える問い

- 括弧の対応は読めるか
- ループは何番か
- -10.20 は大きい? (次で比べる)

# 演習1B：一塩基を変え、構造とエネルギーを比較する

```
import RNA
seq = "GGCGCAGAAAUGCGCC"
pos, new = 4, "A"          # ← ここを変える (位置は 1 始まり)
mut = seq[:pos-1] + new + seq[pos:]
for name, s in [("元の配列", seq), ("変更後", mut)]:
    ss, mfe = RNA.fold(s)
    print(name, s, ss, f"{mfe:.2f}")
print("塩基対距離 =", RNA.bp_distance(RNA.fold(seq)[0],
    RNA.fold(mut)[0]))
```

| 変更     | 予測構造            | $\Delta G$ | bp距離 |
|--------|-----------------|------------|------|
| (元)    | (((((...))))))  | -10.2      | 0    |
| 4: G→A | (((.((...)).))  | -4.8       | 1    |
| 8: A→C | (((((...))))))  | -10.2      | 0    |
| 3: C→U | (((((...))))))  | -8.3       | 0    |
| 1: G→A | .(((((...))))). | -7.9       | 1    |

配布スクリプト：exercises/ex1b\_mutate.py (表は期待される結果の例。位置：元→新)

## 変更する入力

- pos (1~16) と new (A/C/G/U)
- 表の4例から1つ選ぶ

## 操作

- スクリプトを実行
- 元と変更後の構造・ $\Delta G$ を並べる

## 確認する出力

- 構造は変わったか
- $\Delta G$  はいくつ変わったか
- bp距離 (構造の差)

## 考える問い

- どこを変えると壊れる？
- どこは変えても平気？
- Hamming距離1でも構造差が0や2になるのはなぜ？



## 録画を再開します : inverse folding の説明

19:45–20:00 目標構造から配列を探す問題と、RNAinverse の探索の仕組み

演習1で見た「配列を変えると構造とエネルギーが変わる」を、逆向きに使います。

# 予測と設計：入力と出力の違い

|      | 予測 (fold)           | 設計 (inverse fold)            |
|------|---------------------|------------------------------|
| 入力   | 配列 $x$              | 目標構造 $T$                     |
| 出力   | 構造 $S$ と $\Delta G$ | 配列 $x$ ( $MFE(x) = T$ になるもの) |
| 答えの数 | 一つに決まる              | 多数あり得る／見つからないこともある           |
| 計算   | 動的計画法（決定的、高速）       | 予測を繰り返す探索（乱数に依存）             |
| 評価   | エネルギーの最小化           | 予測構造と目標の距離 $d$ を最小化          |

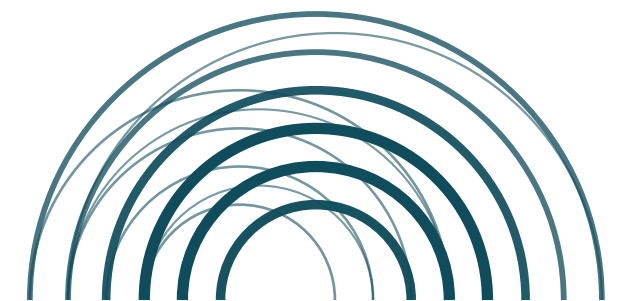
演習1B でやったこと（1 塩基変えて再予測）を、目標に近づく方向へ自動で繰り返すのが設計

## 同じ評価器

- 設計は予測プログラムを何度も呼ぶ
- 「少し変えて、再予測して、目標に近づいたか見る」の繰り返し
- 目標に届いたら終了。届かなければ「失敗」として距離を返す

# 配列設計の難しさ：目標以外の構造との競合

RNAinverse の出力例（MFE 構造は目標に一致）



GGUCGGGAGGUCGGUC

(((((((...)))))))

$\Delta G = -1.1$  kcal/mol      目標構造の確率  $P(T) = 0.232$

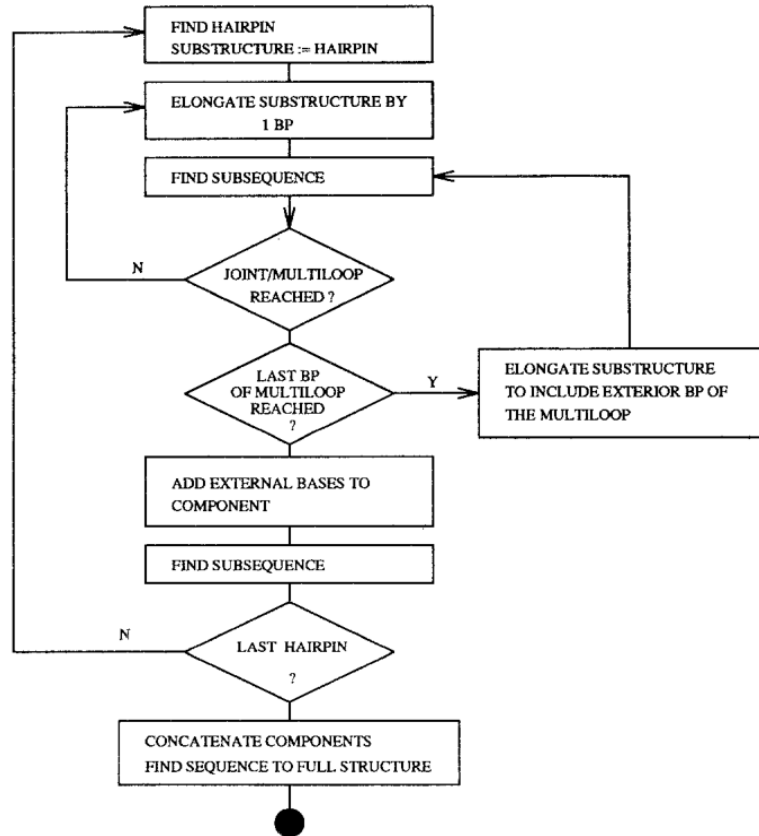
| 構造                | $\Delta G$ | P            |
|-------------------|------------|--------------|
| (((((((...))))))) | -1.1       | <b>0.232</b> |
| ..(((...)))..     | -1.0       | 0.197        |
| ...(((...)))...   | -0.9       | 0.168        |
| (((((.....))))))  | -0.2       | 0.054        |
| (((((...)))....   | -0.1       | 0.046        |

同じ配列がとりうる構造（上位5つ）。目標（1行目）の確率は 23% にすぎない

## 設計で起こること

- 目標以外の構造がほぼ同じエネルギーで存在すると、MFE が一致しても分子は揺らぐ
- MFE の一致 ( $d = 0$ ) と高い目標確率は別の条件（演習2D で確かめる）
- 対策例：目標構造の確率を目的関数にする（RNAinverse の -Fp、inverse\_pf\_fold）

# 元論文の考え方：Hofacker et al. (1994) の inverse folding



- 候補は「適合配列」だけ：目標の塩基対の位置で対合できる文字をもつ配列
- コスト＝構造距離  $f(I) = d(S(I), T)$ 。距離の選び方は性能に大きく影響しない（論文）。現行実装は塩基対距離
- 部分構造から順に：ヘアピン → 1 塩基対ずつ伸ばす → 分岐で結合 → 全長（Fig. 5）。全長の fold 回数を減らし、局所解に陥りにくくする
- 最適化は最も単純な適応的探索：ランダム変異（非対合は 1 塩基、対合は 2 塩基同時）を、コストが下がれば採用。改善なしで停止し、新しい初期配列からやり直す
- 「正しく対合していない位置に限定」する選択枝（現行実装の既定）
- 分配関数版：目標構造の確率  $P(T) = \exp(-\Delta G(T)/RT)/Q$  を最大化（式 6）。全長で最適化するため遅い

Hofacker et al. (1994) Monatsh. Chem. 125:167–188, Fig. 5 「Flow chart of the inverse folding algorithm」

| Fig. 7 の例 (tRNA-Phe、76 塩基) | P(目標構造)     |
|----------------------------|-------------|
| -Fm (MFE 目的) で得た 3 配列      | 0.03 ~ 0.09 |
| -Fp (確率目的) で得た 3 配列        | 0.84 ~ 0.86 |
| 天然の tRNA-Phe 配列            | 0.17        |

# RNAinverseの探索（現行実装）：変異、再予測、評価、更新

## ① 整合化

目標の各塩基対が対合できる文字にする

## ② 予測

MFE 構造を計算し、目標との塩基対距離  $d$  を求める

## ③ 変異

食い違う位置（と隣）を選ぶ。非対合→1塩基、対合→ペアごと

## ④ 評価・更新

$d$  が減れば採用。同点ならエネルギー差が縮む変異を記憶

## ⑤ 終了

$d = 0$  で成功。改善なしなら停止 ( $d > 0$  を返す)

トレース例（目標（((((..(((....)))..))))）、教材用の簡略再現）

| 手順  | 配列                       | 予測構造                       | d  | 変更                            |
|-----|--------------------------|----------------------------|----|-------------------------------|
| 初期  | UCGCAUUUAGCCCCAAUAGCGACU | (((((.....)))))..          | 12 | 乱数で作った初期配列                    |
| 整合化 | UGUCGUUUUCCCCAAGAGCGACG  | .((((((((.....)))))))).    | 3  | 位置 24,2,3,5,17,9,10 を対合できる文字に |
| 更新1 | UGUCGUUUUCCCCAAGAGCGACA  | ((((((((((.....)))))))))   | 2  | ペア(1,24): UG→UA               |
| 更新2 | UGUCGUUUUCCCCAAGAGCGACA  | (((((((.(((.....))).)))))) | 1  | 位置7: U→A                      |
| 更新3 | UGUCGAAUUUCCCCAAGAGCGACA | ((((((..(((....)))..))))   | 0  | 位置6: U→A                      |

繰り返し

$d$  = 目標との塩基対距離（次のスライド）。1回の更新で1~2文字が変わり、 $d$ が単調に減って0に達している

実装も論文どおり、目標をヘアピン側の部分構造から順に解く。成功しても「目標構造の確率が高い」とは限らない（スライド 26）。

# 目標への近さ：塩基対距離の具体例

$$d_{bp}(S, T) = |S \setminus T| + |T \setminus S|$$

目標 T に対し、構造 S の「片方にだけある塩基対」の数。  
RNA.bp\_distance(S, T)

| 構造 S                | 塩基対 | d_bp(S, T) | メモ                  |
|---------------------|-----|------------|---------------------|
| ((((((.....))))))   | 6   | 0          | 目標そのもの              |
| .((((((.....))))).  | 5   | 1          | 末端の1対が外れた           |
| ((((.((.....)).)))  | 5   | 1          | 4番と13番の対が外れた（内部ループ） |
| ..((((((.....)))).. | 4   | 2          | 末端の2対が外れた           |
| ((((((.....))))))   | 5   | 1          | ループ側の1対が外れた         |
| .....               | 0   | 6          | 開いた鎖                |

## Hamming 距離（配列の差）

GGCGCAGAAAUGCGCC  
GGCACAGAAAUGCGCC  
Hamming 距離 = 1

GGCGCAGAAAUGCGCC  
CGCGGAAACGUUCGUG  
Hamming 距離 = 8

設計の評価は「構造の距離」。RNAinverse の出力に添えられる数値は「初期配列からの Hamming 距離」なので混同しない

文字が違う位置の数。構造の差（塩基対距離）とは別に数える

# 探索結果の読み方：成功判定、失敗、初期値依存

| seed | 初期配列             | 設計配列             | Hamming | 再予測              | d | P(T) |
|------|------------------|------------------|---------|------------------|---|------|
| 1    | UCCAGAAACGAUCAUU | CGCGGAAACGUUCGUG | 6       | (((((.....)))))) | 0 | 0.62 |
| 2    | GUGAGCUUCUAAUGAG | CUCAGUUUCUAUUGAG | 4       | (((((.....)))))) | 0 | 0.86 |
| 3    | CCACUGUGCACACCGC | CCAGUGUGCACACUGG | 3       | (((((.....)))))) | 0 | 0.94 |
| 4    | AUCCAGUCACCCUUUC | AUGCGGUCACCCGUGU | 5       | (((((.....)))))) | 0 | 0.51 |
| 5    | UCGCAUUUAGCCCCAA | UCGGAUUUAGGUCCGA | 4       | (((((.....)))))) | 0 | 0.57 |

## 失敗の例（別の目標構造）

- 目標 ((((.((.....)).)))
- 返り値  $d = 1$  (40 回中 2 回失敗)
- UUUCGUGAUUUUAUGGGA
- (((((.....)))))) ← 再予測は目標と 1 対違う

## 読み方

- $d = 0$  : 成功。ただし  $P(T)$  は 0.5~0.94 とばらつく
- $d > 0$  : 失敗。配列は返るが MFE 構造は目標と異なる
- seed（初期配列）で結果が変わる → 複数回試して候補を集める
- 同じ seed でも Windows と macOS で配列が変わる



## 録画を停止します：演習2（40分）

20:00–20:40 配列設計、再予測、候補比較（グループ演習）

必須：2A→2B→2C（ここまで届けば十分）

時間があれば：2D / 発展問題：グループの裁量で  
各スクリプトは単独で動くので、途中からでも再開できます。

# 演習2A：目標構造を指定し、RNA.inverse\_fold を動かす

```
import RNA
target = "(((((((.....))))))"      # 目標構造（例題RNAと同じ形）
seed = 1                             # ← 変えると別の初期配列になる
RNA.init_rand(seed)
start = RNA.random_string(len(target), "ACGU") # A/C/G/U の乱数配列
start_copy = start.encode().decode() # inverse_fold は start を上書きするので控え
seq, d = RNA.inverse_fold(start, target)
print("初期配列:", start_copy)
print("設計配列:", seq, " 残り距離 d =", d)
```

期待する出力（seed=1、macOS）

初期配列: UCCAGAAACGAUCAUU  
設計配列: CGCGGAAACGUUCGUG 残り距離 d = 0.0

配布スクリプト: exercises/ex2a\_inverse.py

start は関数呼び出しで上書きされる。控え（start\_copy）を必ず取る

## 変更する入力

- target（目標構造）
- seed（1 から）

## 操作

- 初期配列を乱数で作る
- RNA.inverse\_fold を実行

## 確認する出力

- 設計配列
- 残り距離 d（0 なら成功）

## 考える問い

- 初期配列と何文字違う？
- d = 0 は何を保証する？
- 隣の人と同じ配列になった？

Windows では同じ seed でも配列・ $\Delta G$ ・P(T) が上の値と異なります（例 seed=1 : UGAUGUCUUGAUGUCA）。d = 0 なら成功です

## 演習2B：設計配列を再び fold し、目標と比較する

```
ss, mfe = RNA.fold(seq)           # 設計配列を再予測
print(seq)
print(ss, f"{mfe:.2f}")
print(target, "← 目標")
print("目標との塩基対距離 =", RNA.bp_distance(ss, target))
print("初期配列からの Hamming 距離 =",
      sum(a != b for a, b in zip(start_copy, seq)))
```

期待する出力 (seed=1、macOS)

```
CGCGGAAACGUUCGUG
(((((((.....)))))) -3.80
(((((((.....)))))) ← 目標
目標との塩基対距離 = 0
初期配列からの Hamming 距離 = 6
```

配布スクリプト：exercises/ex2b\_refold.py（単独で実行可。2A の計算も中でやり直す）

### 変更する入力

- 2A で得た seq と target

### 操作

- RNA.fold(seq)
- RNA.bp\_distance(ss, target)
- Hamming 距離を数える

### 確認する出力

- 再予測の構造と  $\Delta G$
- 塩基対距離 (0 か)
- Hamming 距離

### 考える問い

- $\Delta G$  は例題RNAの  $-10.2$  と比べてどうか
- 「成功」の基準に安定さは入っている？

Windows では同じ seed でも配列・ $\Delta G$ ・P(T) が上の値と異なります（例 seed=1 : UGAUGUCUUGAUGUCA）。d = 0 なら成功です

# 演習2C：複数回設計し、異なる配列の候補を集める

```
results = []
for seed in range(1, 6):          # 5 回だけ
    RNA.init_rand(seed)
    start = RNA.random_string(len(target), "ACGU")
    seq, d = RNA.inverse_fold(start, target)
    ss, mfe = RNA.fold(seq)
    results.append((seed, seq, ss, mfe, RNA.bp_distance(ss, target)))
for r in results:
    print(r[0], r[1], r[2], f"{r[3]:.2f}", "d =", r[4])
```

| seed | 設計配列             | $\Delta G$ | d |
|------|------------------|------------|---|
| 1    | CGCGGAAACGUUCGUG | -3.80      | 0 |
| 2    | CUCAGUUUCUAUUGAG | -3.90      | 0 |
| 3    | CCAGUGUGCACACUGG | -7.30      | 0 |
| 4    | AUGCGGUCACCCGUGU | -4.20      | 0 |
| 5    | UCGGAUUUAGGUCCGA | -6.00      | 0 |

期待する出力（macOS。再予測はすべて目標と一致） 配布：  
exercises/ex2c\_repeat.py

## 変更する入力

- seed の範囲（1～5）
- 余裕があれば 6～10

## 操作

- for ループで 5 回設計
- 各回を再予測して記録

## 確認する出力

- 5 つの配列（全部違う）
- $\Delta G$  のばらつき
- d（失敗があれば > 0）

## 考える問い

- どの候補が「良い」？
- $\Delta G$  が低い＝良い、と言える？
- 失敗候補は何が違う？

Windows では同じ seed でも配列・ $\Delta G$ ・P(T) が上の値と異なります（例 seed=1 : UGAUGUCUUGAUGUCA）。d = 0 なら成功です

# 演習2D : 目標構造の確率を計算し、候補を比較する

```
def target_probability(seq, target):
    fc = RNA.fold_compound(seq)          # 同じモデル条件 (37 °C)
    ss, mfe = fc.mfe()
    fc.exp_params_rescale(mfe)
    fc.pf()                             # 分配関数
    return fc.pr_structure(target)       # 目標構造の確率 P(T)

for seed, seq, ss, mfe, d in results:
    print(seed, seq, f"P(目標) = {target_probability(seq, target):.3f}")
```

| seed | 設計配列             | d | P(T)         |
|------|------------------|---|--------------|
| 1    | CGCGGAAACGUUCGUG | 0 | <b>0.616</b> |
| 2    | CUCAGUUUCUAUUGAG | 0 | <b>0.863</b> |
| 3    | CCAGUGUGCACACUGG | 0 | <b>0.935</b> |
| 4    | AUGCGGUCACCCGUGU | 0 | <b>0.506</b> |
| 5    | UCGGAUUUAGGUCCGA | 0 | <b>0.570</b> |
| -Fp  | GGCGGCGCAGGCCGCC | 0 | <b>0.976</b> |

最終行 : RNA.inverse\_pf\_fold (確率を最大化する版) の結果  
。配布 : exercises/ex2d\_probability.py

## 変更する入力

- 2C の results
- (発展) inverse\_pf\_fold も試す

## 操作

- target\_probability で各候補の P(T)
- 表にして並べる

## 確認する出力

- P(T) : 0.5~0.94 とばらつく
- d = 0 でも低い候補がある

## 考える問い

- MFE 一致と高い P(T) は同じ条件?
- P(T) が高い配列の特徴は?
- GC が多い配列の懸念は?

Windows では候補の配列と P(T) が上の値と異なります。inverse\_pf\_fold の P(T) = 0.976 は共通です

# グループで試す発展問題：目標や配列条件を一つ変えて比較する

## A. 目標構造を変える

- 内部ループ入り (24)
- ((((((...(((.....))))..))))))
- 2 ヘアピン (26)
- (((((.....))))..(((.....))))
- 3 分岐 (53 塩基、難しい)
- → ex3\_group.py のコメント参照

問い：成功率と P(T) はどう変わるか

## B. 塩基を固定する

- 初期配列の一部を小文字にすると固定される
- `start = start[:6] + "gaaa" + start[10:]`
- ループを GAAA に固定して 5 回設計

問い：固定しても成功するか。P(T) は？

## C. 温度を変える

- `RNA.md()` の `temperature` を 25 / 60 に
- 同じ候補の P(T) を再計算
- (設計を別温度で行うなら `RNA.cvar.temperature`)

問い：温度で目標構造の確率はどう変わるか

## 記録すること（提出課題の材料）

- 変えた条件／目標構造／設計配列（複数）／再予測結果／P(T)／気づいたこと



## 録画を再開します：まとめ

20:40-20:50 応用への展開、まとめ、提出課題

グループの結果は録画停止中に共有してもらいました。ここからは本日の内容を整理します。

# 実際の応用で追加する条件：複数状態、相互作用、実験検証

## 複数状態

- スイッチは OFF/ON の両構造を評価
- 複数目標の設計 (RNAblueprint など)

## 相互作用

- トリガーとの二本鎖形成を評価
- RNAcofold / RNAup、NUPACK

## 配列制約

- RBS・AUG・プロモーターの固定
- GC 含量、反復配列や制限部位の回避

## 実験検証

- 予測 = 機能ではない
- 多数設計して実測で選抜 (Green 2014 は 646 設計)

今日の道具（単一状態・二次構造・MFE と確率）は出発点。「MFE 構造が一致すれば実験でも機能する」とは言えない。

# 今日のまとめ：何を入力し、何を計算し、何がまだ分からないか

| 操作                | 入力            | 計算                    | 出力                          | まだ分からないこと                |
|-------------------|---------------|-----------------------|-----------------------------|--------------------------|
| RNA.fold          | 配列            | ループ単位のエネルギー和を DP で最小化 | MFE 構造、 $\Delta G$          | 他の構造がどれだけ存在するか           |
| pf / pr_structure | 配列 (+ 目標構造)   | 全構造のボルツマン和 (分配関数)     | 構造確率 $P(S)$ 、塩基対確率 $p_{ij}$ | モデル外の相互作用・三次構造           |
| RNA.inverse_fold  | 目標構造 (+ 初期配列) | 変異と再予測を繰り返す探索         | 配列と残り距離 $d$                 | 初期値依存、 $P(T)$ の高さ、実験での機能 |
| inverse_pf_fold   | 目標構造          | 目標構造の確率を最大化する探索       | GC の多い安定配列                  | 実験・細胞内での扱いやすさ            |

## 覚えて帰ること

- 配列 → 構造は一意に予測できるが、分子は構造集団として揺らぐ (MFE と確率を区別)
- 構造 → 配列は探索で、答えは複数・初期値依存・失敗もある
- 成功の基準 ( $d = 0$ ) と良さの基準 ( $P(T)$ 、実験) は別
- 資料と演習スクリプトは [github.com/Shunsuke-1994/Course\\_RNAinverse](https://github.com/Shunsuke-1994/Course_RNAinverse) に置いてあります。講習後も試せます。質問は Issues へ

# グループ提出課題：設計結果と比較、限界の考察

## 内容（各項目 2～4 行で十分）

- 目標構造（dot-bracket）と、選んだ理由
- 設計配列（複数可）と再予測結果（構造・ $\Delta G$ ・塩基対距離）
- 比較した条件（seed、目標の変更、固定塩基、温度など）と  $P(T)$
- 考察：うまくいった点、失敗や低確率の候補、今日の方法の限界
- 記録：ViennaRNA のバージョン、温度、seed、OS

## 形式（提案）

- A4 1 枚 または スライド 2 枚
- PDF で 1 グループ 1 件
- グループ名と参加者名を記載

## 期限・提出先

- 期限：講習の約 2 週間後、10 月 22 日（案）
- 提出先・方法：未確定。決まり次第、主催者から案内
- 質問は当日の質疑またはグループの TA へ



## 録画を停止します：質疑応答（10分）

20:50–21:00 質疑応答

本日の内容への質問のほか、「自分ならどのようなRNAを設計したいか」を話してください。

# 質疑応答：自分ならどのようなRNAを設計したいか

## 目標は何か

- 結合・切替・触媒・形。目標二次構造をどう決める？

## 入力は何か

- トリガーRNA、小分子、温度。2 状態が要る？

## 条件は何か

- 固定配列、GC 含量、長さ、他分子との相互作用

## どう確かめるか

- 予測 (P(T)) → 実験 (ゲル、蛍光、AFM)

質問は口頭でもチャットでも。時間内に答えきれない質問はグループの TA と後日共有します。  
講義後の質問は GitHub の Issues で受け付けます：[github.com/Shunsuke-1994/Course\\_RNAinverse/issues](https://github.com/Shunsuke-1994/Course_RNAinverse/issues)

# 補足1 : Nussinov 法（塩基対数の最大化）と熱力学モデルの違い

$$N(i, j) = \max \left\{ N(i+1, j), N(i, j-1), N(i+1, j-1) + \delta_{ij}, \max_{i < k < j} [N(i, k) + N(k+1, j)] \right\}$$

$\delta_{ij} = 1$  (i と j が対合できる、 $j-i > 3$ )、0 (それ以外)

|     | G | A | C | G | C | A | G | A | A | A | U | G | C | G | C | C |
|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| G1  |   | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 3 | 4 | 5 | 5 |
| A2  |   |   | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 2 | 2 | 3 | 4 | 4 | 4 |
| C3  |   |   |   | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 2 | 3 | 4 | 4 | 4 |
| G4  |   |   |   |   | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 3 | 3 | 3 | 3 |
| C5  |   |   |   |   |   | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 2 | 2 | 2 | 2 |
| A6  |   |   |   |   |   |   | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 2 |
| G7  |   |   |   |   |   |   |   | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 2 |
| A8  |   |   |   |   |   |   |   |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| A9  |   |   |   |   |   |   |   |   |   | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| A10 |   |   |   |   |   |   |   |   |   |   | 0 | 0 | 0 | 0 | 0 | 1 |
| U11 |   |   |   |   |   |   |   |   |   |   |   | 0 | 0 | 0 | 0 | 1 |
| G12 |   |   |   |   |   |   |   |   |   |   |   |   | 0 | 0 | 0 | 1 |
| C13 |   |   |   |   |   |   |   |   |   |   |   |   |   | 0 | 0 | 0 |
| G14 |   |   |   |   |   |   |   |   |   |   |   |   |   |   | 0 | 0 |
| C15 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   | 0 |
| C16 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

表 :  $N(i, j)$  (上三角)。右上  $N(1, 16)$  が全体の最大塩基対数

## 同じ配列で比べる

GACGCAGAAAUGCGCC

最大塩基対数の構造

$(.(((.(...))))).$

5 対、 $\Delta G = +2.1$  kcal/mol (不安定)

ViennaRNA の MFE 構造

$..((((.....)))).$

4 対、 $\Delta G = -4.6$  kcal/mol

本数が多い  $\neq$  安定。孤立した塩基対や小さいループはエネルギー的に不利

## 補足2 : ViennaRNA の熱力学的 DP (Zuker 型の漸化式)

$$F_{ij} = \min \left\{ F_{i+1,j}, \min_{i < k \leq j} [C_{ik} + F_{k+1,j}] \right\}$$

外部ループ : i が非対合か、i-k の対合で分割

$$C_{ij} = \min \left\{ H(i,j), \min_{i < k < l < j} [C_{kl} + I(i,j;k,l)], \min_{i < u < j} [M_{i+1,u} + M_{u+1,j-1}^1 + a] \right\}$$

i-j が閉じるループ : ヘアピン / 内部ループ (スタッキングを含む) / 多分岐

$$M_{ij} = \min \left\{ \min_{i < u < j} [(u-i)c + C_{uj} + b], \min_{i < u < j} [M_{i,u-1} + C_{uj} + b], M_{i,j-1} + c \right\}$$

多分岐ループの内部 (最初の分岐を含む)

$$M_{ij}^1 = \min \left\{ M_{i,j-1}^1 + c, C_{ij} + b \right\}$$

多分岐ループの内部 (ちょうど 1 本の分岐)

### 読み方

- $H(i,j)$  : ヘアピンのエネルギー、 $I(i,j;k,l)$  : 内部ループ ( $k=i+1, l=j-1$  ならスタッキング)、 $a, b, c$  : 多分岐ループの定数
- 短い区間から埋めていく点は Nussinov と同じ。違いは「何を最小化するか」 (塩基対数ではなくループ単位の自由エネルギー)
- 分配関数 (確率) 版は  $\min \rightarrow$  和、 $+$   $\rightarrow$  積 (McCaskill 1990)

# 補足3：分配関数と構造確率（McCaskill 1990）

$$P(S) = \frac{e^{-E(S)/RT}}{Z}, \quad Z = \sum_{S'} e^{-E(S')/RT}$$

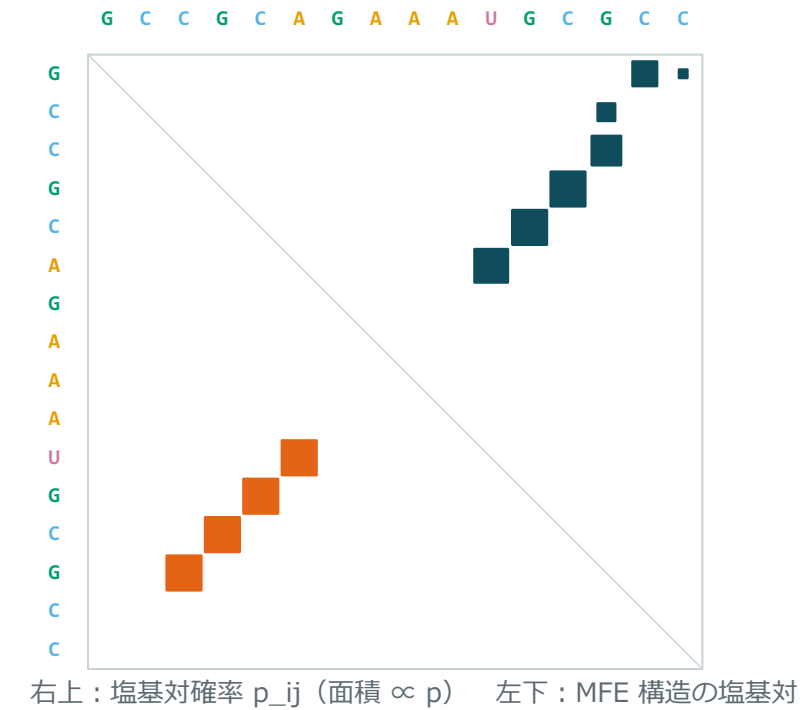
$$p_{ij} = \sum_{S \ni (i,j)} P(S)$$

$$G_{\text{ens}} = -RT \ln Z$$

$$\text{ED}(T) = 1 - \frac{1}{n} \left[ \sum_{(i,j) \in T} p_{ij} + \sum_{i \text{ unpaired in } T} q_i \right]$$

集団欠陥（ensemble defect）：q\_i は i が非対合である確率

ドットプロット：GCCGCAGAAAUGCGCC（2番目を C に変えた配列）



## 例題RNAと G2C 変異体の比較

|                              | 例題RNA              | G2C 変異体                |
|------------------------------|--------------------|------------------------|
| MFE 構造                       | (((((((.....)))))) | ..(((((((.....)))))).. |
| MFE (kcal/mol)               | -10.20             | -4.20                  |
| $G_{\text{ens}} = -RT \ln Z$ | -10.26             | -4.80                  |
| $P(\text{MFE 構造})$           | 0.908              | 0.378                  |
| 集団欠陥 ED(MFE構造)               | 0.012              | —                      |
| 最大の $p_{ij}$                 | 1.000 (2,15)       | 0.998 (5,12)           |

ViennaRNA: fc.pf(), fc.bpp(), fc.pr\_structure(T), fc.ensemble\_defect(T)。RNAfold -p の出力（ドットプロット・mountain plot）は Lorenz et al. 2011 Fig. 1A を参照

## 補足4 : RNAinverse の実装詳細 (v2.7.2 inverse.c)

### MFE モード (inverse\_fold)

- 初期配列の整合化 (make\_start) : 目標の各塩基対が対合できない文字なら片側を置換。小文字側は固定
- 部分構造の利用 (aux\_struct / WALK) : ステム単位に分け、ヘアピン側から外側へ順に adaptive\_walk。give\_up=1 で早期打ち切り
- 変異位置 : 予測と目標が食い違う位置 (非対合または対の 3' 側) → その隣接位置の順。各リストは乱数で並べ替え
- 変異の種類 : 非対合位置は AUGC の他の文字、対合位置は許される塩基対 (AU/UA/GC/CG/GU/UG) をペアごとに置換

### 評価・同点・停止・pf モード

- 評価 :  $\text{cost} = \text{予測構造と目標の塩基対距離 (bp\_distance)}$  。改善した最初の変異を採用 (first improvement)
- 同点時 :  $\text{cost2} = E(\text{目標構造}) - E_{\text{MFE}}$  が小さくなる変異を記憶し、改善がなければ採用
- 停止 :  $\text{cost} = 0$  で成功。改善も同点改善も無ければ停止し、 $\text{cost} (>0)$  を返す
- pf モード :  $\text{cost} = E(\text{目標}) - G_{\text{ens}} = -RT \ln P(\text{目標})$ 。  
 $\text{dangles}=2$  固定、分割探索なし、全非固定位置が候補

### Python API の注意 (2.7.2 で確認)

- `inverse_fold(start, target)` → [配列, cost]。引数 start の文字列オブジェクトが C 側で上書きされる (控えを取る)
- N は置換されない (CLI のみの処理) 。小文字は固定。RNA.cvar.give\_up / symbolset / final\_cost で挙動を変更可

## 補足5 : MFE 目的と確率目的の違い

$$x^* = \arg \min_x d_{bp}(\text{MFE}(x), T)$$

MFE 目的：予測構造と目標の距離を 0 にする

$$x^* = \arg \max_x P(T | x) = \arg \min_x [E(x, T) + RT \ln Z(x)]$$

確率目的：目標構造の確率を最大化

| 目的  | seed | 設計配列               | $\Delta G$ | GC  | P(T)         |
|-----|------|--------------------|------------|-----|--------------|
| MFE | 1    | CGCGGAAACGUUCGUG   | -3.8       | 62% | <b>0.616</b> |
| MFE | 2    | CUCAGUUUCUAUUGAG   | -3.9       | 38% | <b>0.863</b> |
| MFE | 3    | CCAGUGUGCACACUGG   | -7.3       | 62% | <b>0.935</b> |
| MFE | 4    | AUGCGGUCACCCGUGU   | -4.2       | 62% | <b>0.506</b> |
| MFE | 5    | UCGGAUUUAGGUCCGA   | -6.0       | 50% | <b>0.570</b> |
| 確率  | 1    | GGCGGC GCAGGCCGCC  | -12.6      | 94% | <b>0.976</b> |
| 確率  | 2    | GGCGGC UCCGGGCCGCC | -13.1      | 94% | <b>0.976</b> |
| 確率  | 3    | GGCGGC UCCGGGCCGCC | -13.1      | 94% | <b>0.976</b> |

### 読み方

- MFE 目的は  $d = 0$  で止まるため P(T) が低い候補も混じる
- 確率目的は G-C を増やして  $P(T) \approx 0.98$  に到達 (GC 94%)
- GC が多い配列は安定だが、合成・転写・二量体化で扱いにくいことがある
- 目的関数の選択が設計結果を決める
- 原論文 Fig. 7 (tRNA-Phe) でも -Fm は 0.03~0.09、-Fp は 0.84~0.86 と同じ傾向

# 補足6：固定塩基などの制約と温度

## A. 小文字で塩基を固定（ループを gaaa に）

| seed | 初期配列（小文字 = 固定）   | 設計配列             | d | P(T)  |
|------|------------------|------------------|---|-------|
| 1    | UCCAGAgaaaAUCAUU | CGCGGAgaaaUUCGUG | 0 | 0.786 |
| 2    | GUGAGCgaaaAAUGAG | CUCAGUgaaaAUUGAG | 0 | 0.898 |
| 3    | CCACUGgaaaCACCGC | CCAGUGgaaaCACUGG | 0 | 0.946 |

## B. N は置換されない（Python API）

```
RNA.inverse_fold("NNNNNNNNNNNNNNNN", target)
# → GGUAACNNNNGUUAUC   ループの N がそのまま残る
```

## C. 温度と目標構造の確率 P(T)

| 配列                   | 25 °C | 37 °C | 50 °C | 60 °C |
|----------------------|-------|-------|-------|-------|
| GGCGCAGAAUAGCGCC（例題） | 0.90  | 0.91  | 0.84  | 0.84  |
| CGCGGAAACGUUCGUG     | 0.73  | 0.62  | 0.43  | 0.28  |
| CUCAGUUUCUAUUGAG     | 0.91  | 0.86  | 0.68  | 0.49  |
| CCAGUGUGCACACUGG     | 0.95  | 0.94  | 0.85  | 0.80  |
| AUGCGGUCACCCGUGU     | 0.66  | 0.51  | 0.32  | 0.21  |

## コード

```
# 固定：小文字
start = start[:6]+"gaaa"+start[10:]
# 確率計算の温度
md = RNA.md(); md.temperature = 60
fc = RNA.fold_compound(seq, md)
# 設計の温度（旧APIのグローバル）
RNA.cvar.temperature = 60
# 早期打ち切り
RNA.cvar.give_up = 1
GC 含量や反復回避などの制約は RNAinverse にはない
。別ツール（NUPACK, RNAb Blueprint など）や自作の後
処理で対応する
```

# 補足7：実習環境とトラブル対応

```
# 事前準備 (配布手順の要約)
uv init viennarna-course --python 3.13
cd viennarna-course
uv add viennarna==2.7.2
uv run python -c "import RNA; print(RNA.__version__)" # 2.7.2

# 当日の動作確認
uv run python exercises/ex1_fold.py

# 別の方法: GitHub から一式を入手してそのまま使う
git clone https://github.com/Shunsuke-1994/Course_RNAinverse.git
cd Course_RNAinverse && uv sync
uv run python exercises/ex0_check.py
```

## 検証済み・未検証

- 検証済み：macOS 15 (Python 3.13.4、uv 0.12)、Windows 11 (Python 3.13.15、事務局確認)、viennarna 2.7.2
- Windows では同じ seed でも設計配列が変わる (d = 0 なら成功)
- 未検証：他の macOS/Linux 構成
- 動かない参加者は TA・グループの画面で結果比較に参加

| 症状                       | 原因の候補                     | 対応                             |
|--------------------------|---------------------------|--------------------------------|
| ModuleNotFoundError: RNA | 別の Python を起動している         | uv run python を使う／仮想環境を有効化     |
| バージョンが 2.7.2 でない         | 別環境の ViennaRNA            | uv add viennarna==2.7.2 で入れ直す  |
| 同じ seed で配列が違う           | OS ごとの乱数実装の差              | 数値の一致は求めない。d と P(T) の傾向を比べる    |
| start が設計配列に変わっている       | inverse_fold が start を上書き | 控え start.encode().decode() を使う |
| unequal length のエラー      | 配列長と構造長の不一致               | len() で確認。括弧の対応も確認             |
| N が残る                    | Python API は N を置換しない     | A/C/G/U で初期配列を作る               |

## 補足8：参考文献・URL

- 本講習の資料・演習スクリプト（資料 v2.0）：[https://github.com/Shunsuke-1994/Course\\_RNAinverse](https://github.com/Shunsuke-1994/Course_RNAinverse)
- ViennaRNA 公式 <https://www.tbi.univie.ac.at/RNA/>
- Python API [https://viennarna.readthedocs.io/en/latest/api\\_python.html](https://viennarna.readthedocs.io/en/latest/api_python.html)
- RNAinverse マニュアル <https://viennarna.readthedocs.io/en/latest/man/RNAinverse.html>
- ソース: `src/ViennaRNA/inverse/inverse.c` (v2.7.2, <https://github.com/ViennaRNA/ViennaRNA>)
- Hofacker I.L. et al. (1994) Fast folding and comparison of RNA secondary structures. *Monatsh Chem* 125:167–188. <https://doi.org/10.1007/BF00818163>
- Lorenz R. et al. (2011) ViennaRNA Package 2.0. *Algorithms Mol Biol* 6:26. <https://doi.org/10.1186/1748-7188-6-26>
- Zuker M., Stiegler P. (1981) *Nucleic Acids Res* 9:133–148 (熱力学的 DP)
- McCaskill J.S. (1990) *Biopolymers* 29:1105–1119 (分配関数アルゴリズム)
- Nussinov R. et al. (1978) *SIAM J Appl Math* 35:68–82 (塩基対数最大化)
- Mathews D.H. et al. (2004) *PNAS* 101:7287–7292 (Turner 2004 パラメータ); Turner & Mathews (2010) *NNDB, Nucleic Acids Res* 38:D280. <https://rna.urmc.rochester.edu/NNDB/>
- Green A.A., Silver P.A., Collins J.J., Yin P. (2014) Toehold switches: de-novo-designed regulators of gene expression. *Cell* 159:925–939. <https://doi.org/10.1016/j.cell.2014.10.002> (PMC4265554)
- Geary C., Rothmund P.W.K., Andersen E.S. (2014) A single-stranded architecture for cotranscriptional folding of RNA nanostructures. *Science* 345:799–804. <https://doi.org/10.1126/science.1253920>
- Hammer S., Tschitschek B., Flamm C., Hofacker I.L., Findeiß S. (2017) *RNAblueprint. Bioinformatics* 33:2850–2852
- Zadeh J.N. et al. (2011) NUPACK: analysis and design of nucleic acid systems. *J Comput Chem* 32:170–173
- PyPI viennarna <https://pypi.org/project/ViennaRNA/> (使用: 2.7.2)
- uv <https://docs.astral.sh/uv/>